

Tecnatura Superior en Desarrollo de Software

Módulo 07 — Capa InterfaceAdapter: Presenters

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

Prerequisitos de lectura

Antes de avanzar con este módulo, se recomienda haber leído:

Anexo 07a — `OperationResult<T>`

Cubre: definición, estados posibles (Ok/Fail), constructor privado, métodos estáticos, flujo con `OutputPort`, errores comunes.

Anexo 07b — Jerarquía completa del `OutputPort`

Cubre: evolución del `OutputPort` simple (Módulo 04) a la jerarquía de 4 niveles, `IBaseOutputPort<T>`, interfaz específica del UseCase, `BasePresenter<T>`, `Presenter` concreto, diagrama de relaciones.

Este módulo retoma esas bases y las integra en la capa **InterfaceAdapter**: cómo se construye el puente entre la lógica de aplicación y el mecanismo de presentación, qué responsabilidades tiene cada tipo de clase en esa capa, y cómo se registra todo en el contenedor de DI.

¿Qué es la capa InterfaceAdapter?

En la arquitectura limpia, la capa **InterfaceAdapter** ocupa el tercer anillo — entre la lógica de aplicación (UseCase/Ports) y el mundo exterior (frameworks, UI, APIs). Su único propósito es *convertir*: toma los datos que vienen del mundo exterior y los transforma en algo que el UseCase puede entender (Request DTO), y toma los resultados que el UseCase produce y los transforma en algo que el mundo exterior puede mostrar (el resultado que expone el Presenter).

La palabra clave es **adaptador**: esta capa no tiene lógica de negocio, no valida reglas de dominio, no accede a bases de datos. Solo traduce. Si el dominio cambia su forma de comunicar errores, solo la capa InterfaceAdapter debe ajustarse — el UseCase permanece intacto. Si la UI migra de Blazor a una API REST, solo cambia esta capa — el UseCase no sabe nada de la interfaz.

InterfaceAdapter

El anillo traductor de Clean Architecture

Anillo 3 de 4

La capa InterfaceAdapter contiene los tipos que conectan los puertos de la capa Application con los mecanismos concretos de presentación o transporte de datos. No tiene dependencias hacia afuera — solo conoce las interfaces definidas en Application.

Sus responsabilidades son estrictamente tres: recibir la petición del exterior, invocar el UseCase correspondiente a través del InputPort, y capturar el resultado a través del OutputPort para traducirlo al formato requerido por el consumidor.

Tipos típicos:

- **Presenter** — implementa el OutputPort específico del UseCase
- **BasePresenter<T>** — clase base abstracta, centraliza los 3 canales comunes
- **DependencyContainer** — registra Presenter y OutputPort en DI
- **ViewModel** — orquesta el flujo (se construye en el módulo de consumo — Módulo 07b, MVVM en Blazor Hybrid)

Regla de dependencia:

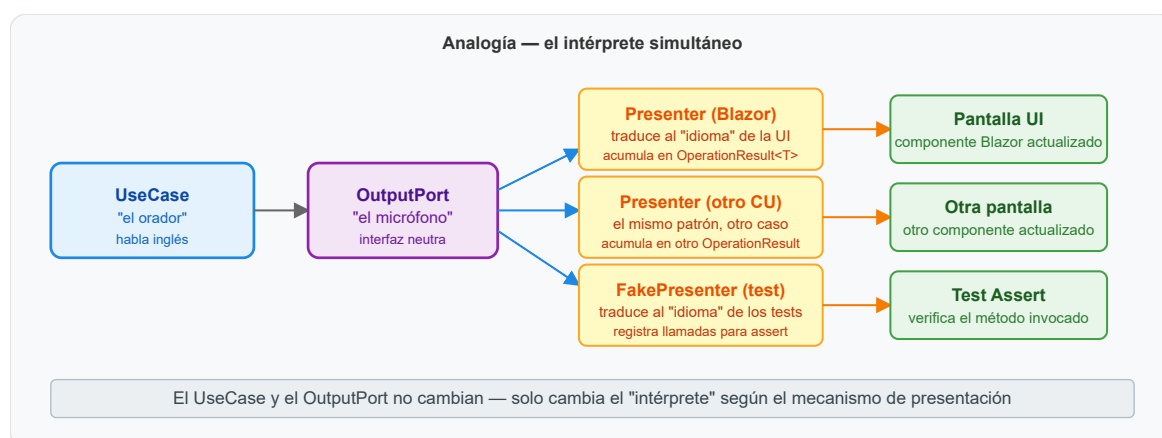
La capa InterfaceAdapter *conoce* Application (sus interfaces, DTOs y puertos), pero Application *no conoce* InterfaceAdapter. El flujo de dependencias siempre apunta hacia adentro — hacia el dominio. Esta es la regla de dependencia de Clean Architecture.

Analogía del mundo real

Imaginá el servicio de interpretación simultánea en una conferencia internacional. El orador (el UseCase) habla en inglés — su idioma propio, sin importarle quién escucha. Los intérpretes (los InterfaceAdapters) escuchan cada frase del orador y la traducen en tiempo real: uno al español, otro al francés, otro al chino.

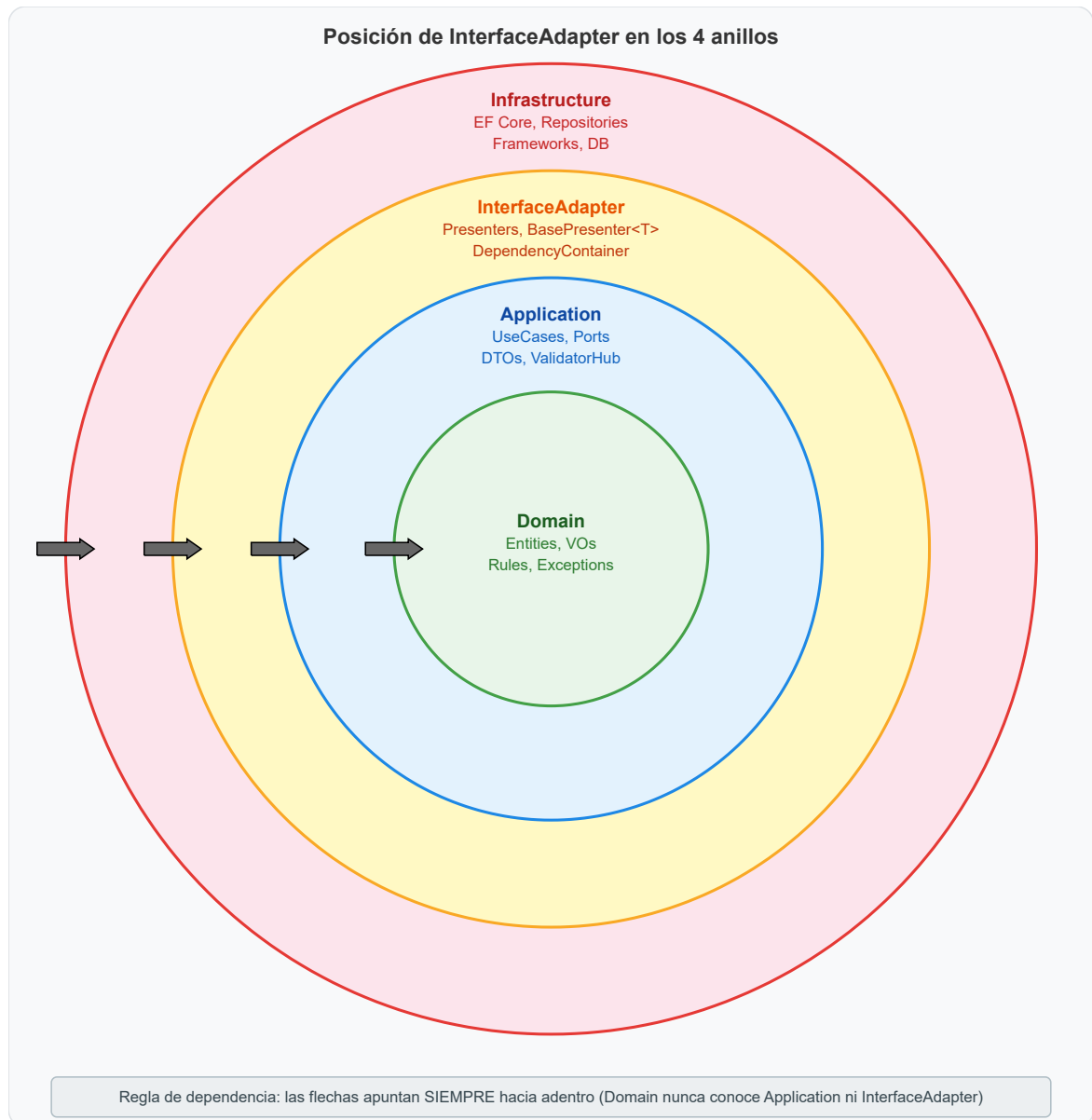
El orador nunca sabe cuántos intérpretes hay ni qué idiomas hablan. Los intérpretes no modifican el mensaje — solo lo traducen. Si el congreso decide agregar un nuevo idioma, se suma un nuevo intérprete; el orador no cambia nada.

En el código: el UseCase es el orador, el OutputPort es el micrófono, el Presenter es el intérprete que escucha a través del OutputPort y acumula el resultado — listo para que la pantalla lo muestre o el test lo verifique.



¿Cómo encaja en la arquitectura?

El diagrama de anillos de Clean Architecture muestra cuatro capas concéntricas. La capa InterfaceAdapter es el tercer anillo: conoce Application (hacia adentro) pero no es conocida por ella. El cuarto anillo —Infraestructura— contiene los frameworks y el acceso a datos; no pertenece a esta capa.



En el proyecto CommerX, la capa InterfaceAdapter vive en el proyecto **CommerX.InterfaceAdapter**. Su estructura interna refleja los mismos agregados del dominio: una carpeta **Customers/** con los presenters y el contenedor de DI propios del agregado Customer.

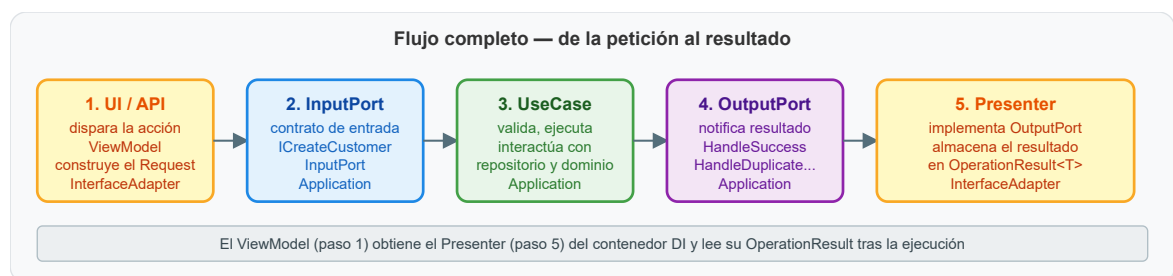
Estructura de carpetas — CommerX.InterfaceAdapter:

```
CommerX.InterfaceAdapter/  
  Customers/  
    Presenters/
```

CreateCustomerPresenter.cs
DependencyContainer.cs

El flujo completo de una petición

Entender cómo fluye una petición a través de las capas es clave para comprender el rol de InterfaceAdapter. El flujo tiene cinco pasos que siempre se suceden en el mismo orden:



Observá que el paso 1 y el paso 5 son ambos responsabilidad de InterfaceAdapter: el ViewModel construye el Request y lanza el UseCase, y el Presenter recibe las notificaciones del OutputPort y construye el resultado final (`OperationResult<T>`). El ViewModel no ejecuta lógica: solo *orquesta*.

Tipos principales de la capa

El Presenter

El **Presenter** es la clase concreta de InterfaceAdapter que implementa el OutputPort específico del UseCase. Cada método del OutputPort que el UseCase puede llamar (`HandleSuccessAsync`, `ValidationErrorsAsync`, `HandleErrorAsync` — los tres heredados de la base — más los métodos propios de este UseCase, como `HandleDuplicateISBNAsync`) tiene su implementación en el Presenter.

El Presenter hereda de `BasePresenter<T>`, que ya implementa los tres métodos comunes de la interfaz base `IBaseOutputPort<T>` almacenando el resultado en un `OperationResult<T>`. De este modo el Presenter solo

implementa los métodos propios de su UseCase.

De dónde viene esta jerarquía:

en Módulo 04 el OutputPort era una interfaz plana: un método por cada resultado posible, sin ningún contrato compartido con el OutputPort de otro UseCase. A medida que el sistema crece, esa repetición se vuelve costosa — por eso acá se extrae lo común (éxito, errores de validación, error de dominio) a `IBaseOutputPort<T>` y cada OutputPort específico solo declara lo que le es propio. El desarrollo completo de esa evolución, con analogía y errores comunes, está en el **Anexo 07B**.

```
1 // ejemplo con dominio neutro: registro de libro
2 public sealed class RegisterBookPresenter
3     : BasePresenter<RegisterBookResponse>,
4       IRegisterBookOutputPort
5 {
6     // este método es específico de este OutputPort – no está en la base
7     public Task HandleDuplicateISBNAsync(string isbn)
8     {
9         // construye un resultado fallido con mensaje descriptivo
10         _result = OperationResult<RegisterBookResponse>.Fail($"ISBN duplicado: {i
11         return Task.CompletedTask;
12     }
13 }
```

Los métodos heredados de `BasePresenter<T>` ya están implementados: `HandleSuccessAsync`, `ValidationErrorsAsync` y `HandleErrorAsync` se encargan de construir el `OperationResult<T>` correspondiente. El Presenter solo completa lo que le falta.

BasePresenter<T>

`BasePresenter<T>` es una clase abstracta que implementa `IBaseOutputPort<T>`. Centraliza el comportamiento común a todos los presenters: guardar el resultado en `_result` usando `OperationResult<T>`. Sus métodos son `virtual` para permitir que subclases los sobrescriban si necesitan comportamiento adicional.

```
1 // clase base abstracta – reutilizable para cualquier UseCase
```

```
2 public abstract class BasePresenter<T>
3     : IBaseOutputPort<T>
4 {
5     // campo protegido – las subclases pueden leerlo
6     protected OperationResult<T>? _result;
7
8     // acceso de solo lectura al resultado acumulado
9     public OperationResult<T>? Result => _result;
10
11     public virtual Task HandleSuccessAsync(T response)
12     {
13         // resultado exitoso – almacena el DTO de respuesta
14         _result = OperationResult<T>.Ok(response);
15         return Task.CompletedTask;
16     }
17
18     public virtual Task ValidationErrorsAsync(IReadOnlyList<ValidationError> errors)
19     {
20         // resultado fallido – almacena los errores de validación del DTO
21         _result = OperationResult<T>.Fail(errors.Select(e => e.Message).ToList());
22         return Task.CompletedTask;
23     }
24
25     public virtual Task HandleErrorAsync(string message)
26     {
27         // resultado fallido – almacena un error de string (duplicado, dominio)
28         _result = OperationResult<T>.Fail(message);
29         return Task.CompletedTask;
30     }
31 }
```

El ViewModel — el orquestador

El ViewModel es el orquestador que consume el Presenter desde afuera de InterfaceAdapter. Esta aplicación es Blazor Hybrid corriendo dentro de un host de escritorio WPF — no hay API REST ni Controllers en ningún punto del curso. El ViewModel no tiene lógica de negocio. Sigue siempre el mismo patrón de tres pasos: construir el Request DTO, invocar el UseCase a través del InputPort, y leer el resultado acumulado por el Presenter.

Este módulo no construye el ViewModel concreto:

acá alcanza con entender su responsabilidad, porque condiciona cómo se registra el Presenter en el contenedor de DI (próxima sección). La construcción real de esa pieza —Vista + ViewModel en Blazor Hybrid— se desarrolla en el módulo que define el mecanismo de consumo (Módulo 07b — MVVM en Blazor Hybrid).

```
1 // esquema del orquestador – los tres pasos, sin una clase completa todavía
2 await _useCase.ExecuteAsync(request);
3
4 var result = _presenter.Result;
5
6 if (result!.IsSuccess)
7     ShowSuccess(result.Value!);
8 else
9     ShowErrors(result.Errors);
```

El DependencyContainer de InterfaceAdapter

Igual que Application tiene su propio `DependencyContainer.cs` con el "namespace truco", InterfaceAdapter registra sus propias implementaciones en un método de extensión homónimo. El Presenter se registra como `Scoped` y **a la vez** como implementación del OutputPort, para que el contenedor DI inyecte siempre la misma instancia cuando alguien pida el Presenter o el OutputPort en el mismo scope de request.

```
1 // namespace truco – extiende IServiceCollection sin agregar using en Program.cs
2 namespace Microsoft.Extensions.DependencyInjection;
3
4 public static class DependencyContainer
5 {
6     public static IServiceCollection AddBookPresenter(
7         this IServiceCollection services)
8     {
9         // registrar el Presenter como implementación concreta
10        services.AddScoped<RegisterBookPresenter>();
11
12        // registrar el OutputPort apuntando a la misma instancia del Presenter
13        services.AddScoped<IRegisterBookOutputPort>(
14            sp => sp.GetRequiredService<RegisterBookPresenter>());
15    }
```

```
16         return services;
17     }
18 }
```

¿Por qué registrar la misma instancia dos veces?

El UseCase recibe el `IRegisterBookOutputPort` por inyección de constructor. El ViewModel también necesita el `RegisterBookPresenter` para leer `Result` después de la ejecución. Si fueran instancias distintas, el ViewModel leería un Presenter vacío. Al usar `GetRequiredService<RegisterBookPresenter>()` como factory del OutputPort, ambas resoluciones apuntan a la misma instancia dentro del scope.

Marco teórico complementario

El patrón Presenter en Clean Architecture

El patrón **Presenter** tiene sus raíces en el MVP (Model-View-Presenter) clásico, pero en Clean Architecture adquiere un rol más específico: es la clase que recibe notificaciones del UseCase a través del OutputPort y las transforma en una representación adecuada para el consumidor. No muestra nada por sí mismo — solo acumula el resultado. La decisión de qué hacer con ese resultado (navegar, mostrar un mensaje de error, retornar un HTTP 400) pertenece al orquestador (el ViewModel), no al Presenter.

Esta separación permite testear el UseCase en forma aislada: basta con un `Fake Presenter` que implemente el OutputPort, sin necesidad de UI ni base de datos. El test ejecuta el UseCase, luego verifica qué método del FakePresenter fue invocado.

Acoplamiento vs. cohesión en InterfaceAdapter

La capa InterfaceAdapter está diseñada para tener **acoplamiento bajo hacia adentro** (solo conoce las interfaces de Application, no las implementaciones concretas) y **cohesión alta hacia afuera** (cada Presenter o ViewModel es

específico de un caso de uso concreto). Esta combinación hace que los cambios en Infrastructure o en la UI no se propaguen al dominio y viceversa.

Scope de vida y el problema de la instancia compartida

El ciclo de vida `Scoped` garantiza que cada request HTTP (o cada unidad de trabajo) obtenga su propia instancia del Presenter y del UseCase. Si se registraran como `Singleton`, el resultado de un usuario podría sobrescribir el de otro. Si se registraran como `Transient`, el UseCase y el ViewModel obtendrían instancias distintas del Presenter, y el ViewModel leería siempre un resultado vacío. `Scoped` es el único ciclo de vida correcto para esta jerarquía.

Resumen de responsabilidades por tipo:

Presenter — implementa el OutputPort específico; hereda de BasePresenter; almacena el resultado en `OperationResult<T>`.

BasePresenter<T> — implementa `IBaseOutputPort<T>`; centraliza el comportamiento común; usa virtual para permitir override.

ViewModel — orquesta el flujo; inyecta UseCase y Presenter; no tiene lógica de negocio; lee Result después de la ejecución.

DependencyContainer — registra Presenter como Scoped; registra OutputPort apuntando a la misma instancia del Presenter vía factory.

Errores comunes

ERROR COMUN

Poner lógica de negocio en el ViewModel:

```
1 // MAL - el ViewModel toma decisiones de negocio
2 if (request.ISBN.Length != 13)
3     throw new Exception("ISBN inválido");
4 await _useCase.ExecuteAsync(request);
```

El ViewModel no sabe nada de reglas — eso es responsabilidad del Guard o del dominio.

BUENA PRACTICA

El ViewModel solo orquesta — invoca y lee resultado:

```
1 // BIEN - sin lógica en el ViewModel
2 await _useCase.ExecuteAsync(request);
3 var result = _presenter.Result;
4 // actuar según IsSuccess
```

La lógica de validación y de negocio ya fue ejecutada por el UseCase y el Guard.

ERROR COMUN

Registrar el Presenter como Transient:

```
1 // MAL - instancias distintas en el mismo request
2 services.AddTransient<RegisterBookPresenter>();
3 services.AddTransient<IRegisterBookOutputPort,
4     RegisterBookPresenter>();
```

El ViewModel recibirá un Presenter vacío — el UseCase escribió en otra instancia.

BUENA PRACTICA

Scoped + factory para compartir instancia:

```
1 // BIEN – misma instancia dentro del scope
2 services.AddScoped<RegisterBookPresenter>();
3 services.AddScoped<IRegisterBookOutputPort>()
4     sp => sp.GetRequiredService<RegisterBookPresenter>());
```

ERROR COMUN

El UseCase conoce el Presenter concreto:

```
1 // MAL – Application depende de InterfaceAdapter
2 using CommerX.InterfaceAdapter.Customers;
3 var vm = new RegisterBookPresenter();
4 vm.HandleSuccessAsync(response);
```

Viola la regla de dependencia — Application nunca debe conocer InterfaceAdapter.

BUENA PRACTICA

El UseCase solo conoce la interfaz del OutputPort:

```
1 // BIEN – depende de la interfaz definida en Application
2 private readonly IRegisterBookOutputPort _output;
3 await _output.HandleSuccessAsync(response);
```

Qué NO hacer

QUE NO HACER

Acceder a Infrastructure desde InterfaceAdapter:

```
1 // NO – el Presenter no debe ir a la base de datos
2 public Task HandleSuccessAsync(RegisterBookResponse r)
3 {
4     _dbContext.AuditLog.Add(...);
5     await _dbContext.SaveChangesAsync();
6 }
```

InterfaceAdapter no debe tener dependencias de frameworks o base de datos. Esas operaciones pertenecen a Infrastructure.

BUENA PRACTICA

El Presenter solo acumula el resultado:

```
1 // BIEN – acumula y delega la presentación
2 public Task HandleSuccessAsync(RegisterBookResponse r)
3 {
4     _result = OperationResult<RegisterBookResponse>.Ok(r);
5     return Task.CompletedTask;
6 }
```